

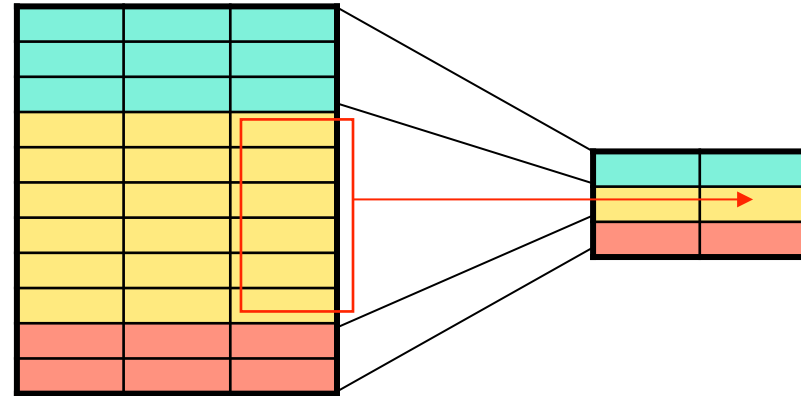
Windowed Table Functions

- Windowed table function
 - operates on a window of a table
 - returns a value for every row in that window
 - the value is calculated by taking into consideration values from the set of rows in that window
- 5 new windowed table functions
 - RANK () OVER ...
 - DENSE_RANK () OVER ...
 - PERCENT_RANK () OVER ...
 - CUME_DIST () OVER ...
 - ROW_NUMBER () OVER ...
- In addition, 8 old aggregate functions and 16 new aggregate functions can also be used as windowed table functions:
Example: `sum(salary) OVER ...`
- Allows calculation of moving and cumulative aggregate values.

Concept (Compared To Set Functions)

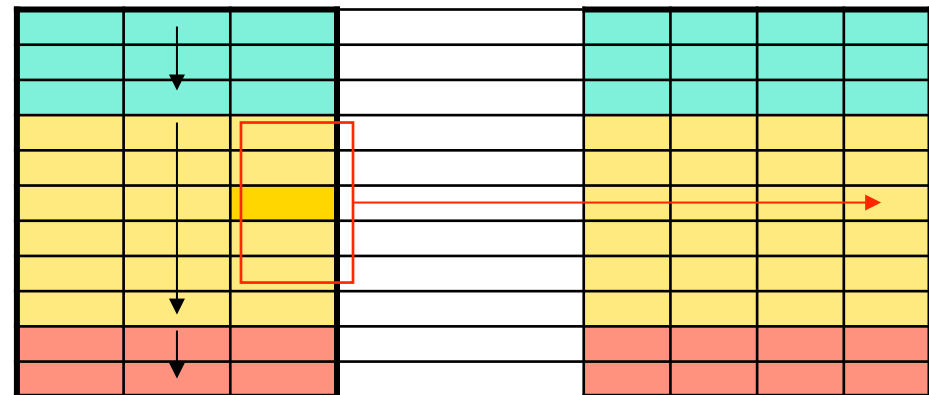
- Set functions
(aggregate functions)

```
SELECT dept, AVG(salary)
FROM Employees
GROUP BY dept
```



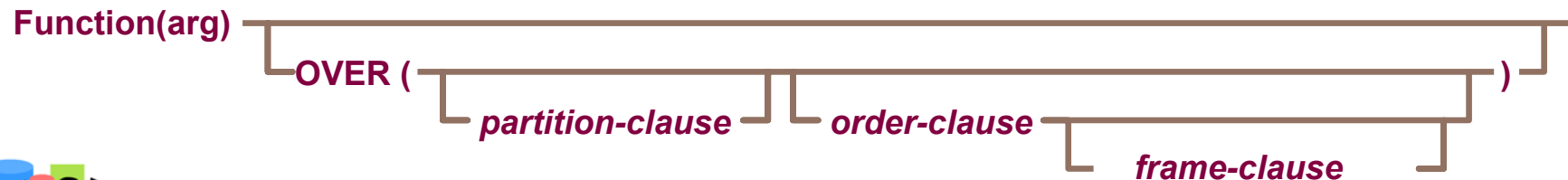
- Windowed Table Functions
(tuple-based aggregation)

```
SELECT dept, empno, salary,
       AVG(salary) OVER(
         PARTITION BY dept
         ORDER BY age
         ROWS
           BETWEEN 2 PRECEDING
             AND 2 FOLLOWING)
FROM Employees
```



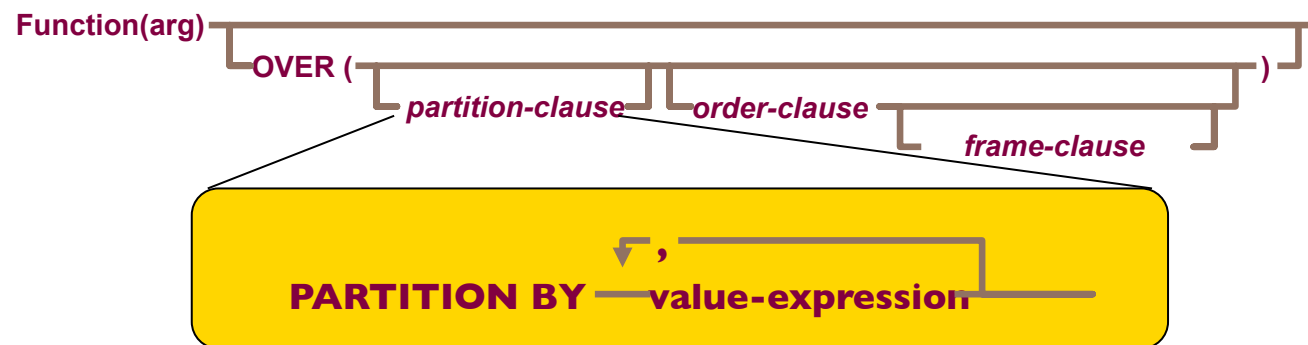
Windowed Tables and Window Functions

- Windowed table
 - table (result of a table expression) together with one or more **windows**
 - windows are independent from each other
- Window
 - defines, for each row in the table, a set of rows (*current row window*) that is used to compute additional attributes
 - specified using a **window specification** (**OVER** clause)
 - based on three main concepts
 - window **partitioning** is similar to forming groups, but rows are retained
 - window **ordering** defines an order (sequence) of rows within each partition
 - window **frame** is defined relative to each row to further restrict the set of rows
- Window **function**
 - is applied for each row, over the current row window, returning a single value
 - used in column expressions in the select-list



The Partitioning Clause

- The partition-clause allows to subdivide the rows into partitions, much like the group by clause



- Without further clauses, the current row window contains all the rows of the same partition (i.e., all the rows that are not distinct from the current row, including the current row)
 - if no partitioning clause is specified, then there is a single partition that contains the complete table
- Windows do not reach across partition boundaries!

Set Functions as Window Functions

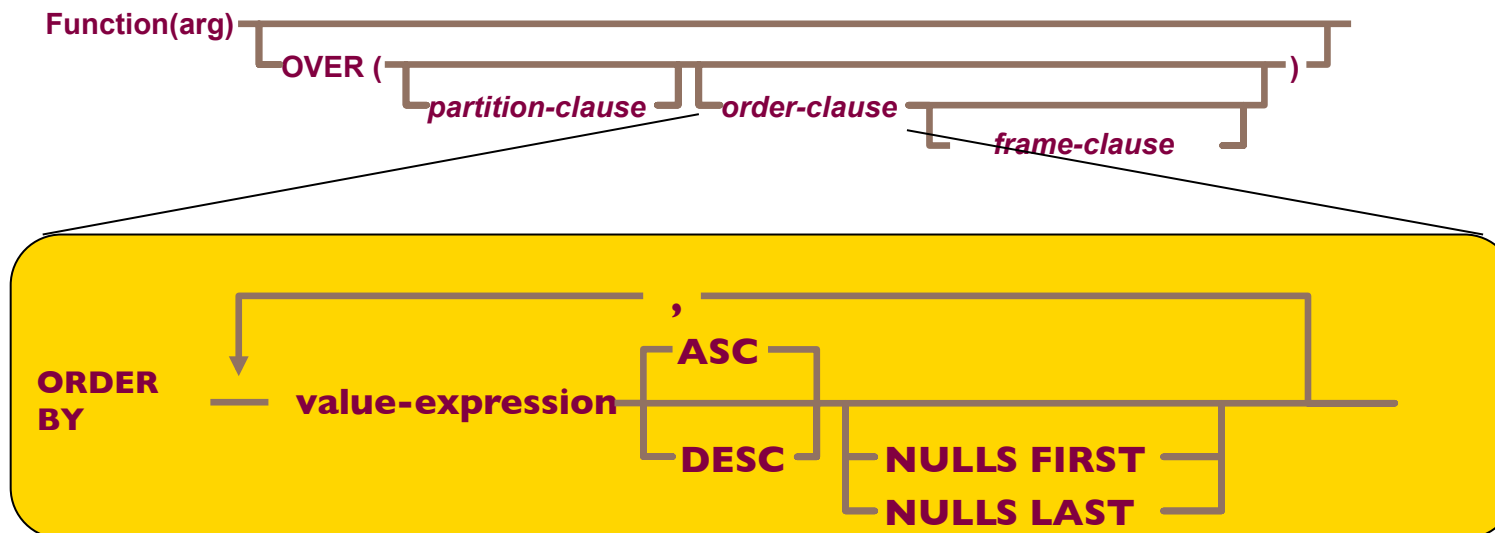
- The OVER clause turns a set function into a window function
 - Aggregated value is computed per current row window (here: per partition)

```
select empnum, dept, salary,  
       avg(salary) over (partition by dept) as dept_avg  
from emptab;
```

EMPNUM	DEPT	SALARY	DEPT_AVG
6	1	78000	63833
2	1	75000	63833
7	1	75000	63833
11	1	53000	63833
5	1	52000	63833
1	1	50000	63833
9	2	51000	51000
4	2	-	51000
8	3	79000	69667
12	3	75000	69667
10	3	55000	69667
3	-	84000	84000
0	-	-	84000

The Order Clause

- The order-clause defines an order (sequence) within a partition
- May contain multiple order items
 - Each item includes a value-expression
 - NULLS FIRST/LAST defines ordering semantics for NULL values
- This clause is completely independent of the query's ORDER BY clause



Ranking Functions For Sequences

- RANK
 - returns the relative position of a value in an ordered group
 - equal values (ties) are ranked the same
- DENSE_RANK
 - like RANK, but no gaps in rankings in the case of ties
- ROW_NUMBER
 - ties are non-deterministically numbered
- Ordering is required!
- Example:

```
select empnum, dept, salary,  
       rank() over (order by salary desc nulls last) as rank,  
       dense_rank() over (order by salary desc nulls last) as denserank,  
       row_number() over (order by salary desc nulls last) as rownum  
from emptab;
```

Ranking Functions Example

EMPNUM	DEPT	SALARY	RANK	DENSERANK	ROWNUM
3	-	84000	1	1	1
8	3	79000	2	2	2
6	1	78000	3	3	3
2	1	75000	4	4	4
7	1	75000	4	4	5
12	3	75000	4	4	6
10	3	55000	7	5	7
11	1	53000	8	6	8
5	1	52000	9	7	9
9	2	51000	10	8	10
1	1	50000	11	9	11
4	2	-	12	10	12
0	-	-	12	10	13

Example: Rank with Ordering and Partitioning

- Find rankings of each employee's salary within her department

```
select empnum, dept, salary,  
       rank() over (partition by dept order by salary desc nulls last)  
         as rank_in_dept,  
       rank() over (order by salary desc nulls last) as globalrank  
from emptab;
```

EMPNUM	DEPT	SALARY	RANK_IN_DEPT	RANK
6	1	78000	1	3
2	1	75000	2	4
7	1	75000	2	4
11	1	53000	4	8
5	1	52000	5	9
1	1	50000	6	11
9	2	51000	1	10
4	2	-	2	12
8	3	79000	1	2
12	3	75000	2	4
10	3	55000	3	7
3	-	84000	1	1
0	-	-	2	12

Rank on Aggregations

- Windowed table functions are computed in the select list
 - After applying FROM, WHERE, GROUP BY, HAVING
 - They may not be referenced in any of these clauses
 - May use aggregation functions in window specification expressions
 - If you wish to reference them, you must nest them, or use a common table expression
- Example: Find rankings of each department's total salary
select dept, sum(salary) as sumsal,
rank() over (order by **sum(salary)** desc nulls last) as rankdept
from emptab
group by dept;

DEPT	SUMSAL	RANKDEPT
1	383000	1
3	209000	2
-	84000	3
2	51000	4

SQL Query Processing Steps incl. OLAP

